QCon
全球软件开发大会

InfoQ 技术大会

从 App 到 Agent：字节跳动 Supabase 的 Agent-Native 规模化实践

张国强

字节跳动资深技术专家



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景

02 从 App 到 Agent：后端基础设施发生了什么变化

03 字节跳动 Agent-Native BaaS 技术方案

04 字节跳动 Supabase 的规模化实践

05 未来展望

01

背景

AI 给软件开发方式和运行形态带来的变化

开发主体从工程师写代码转向大模型生成应用，后端供给方式必须随之改变

自动化程度 · 交付速度 · 产物规模持续提升

2021 年前

传统编程

工程师手写 / 人工搭建后端

手写

需求 — 代码 — 后端 — 发布

人工设计表结构、编写 DAO

后端占开发量 60%～80%

交付周期

数周至数月

2022—2024

传统编程 + Copilot

AI 辅助编码 / 效率提升

辅助

工程师（主导） — Copilot（补全片段） — 后端（人工设计）

AI 补全片段代码，编码效率明显提升

但后端架构仍需人工

代码更快，供给方式未变

2025 至今

Vibe Coding

大模型生成应用 / 全栈交付

生成

产物量：千级 → 千万级

大模型（直接生成完整应用）

- 应用 A · 全栈
- 应用 B · 全栈
- 应用 N · 全栈

开箱即用的后端供给

Auth | Database | Storage | Functions | Realtime

结论

开发瓶颈正在从写代码，转向后端能力能否被模型即时发现、组合与运行。

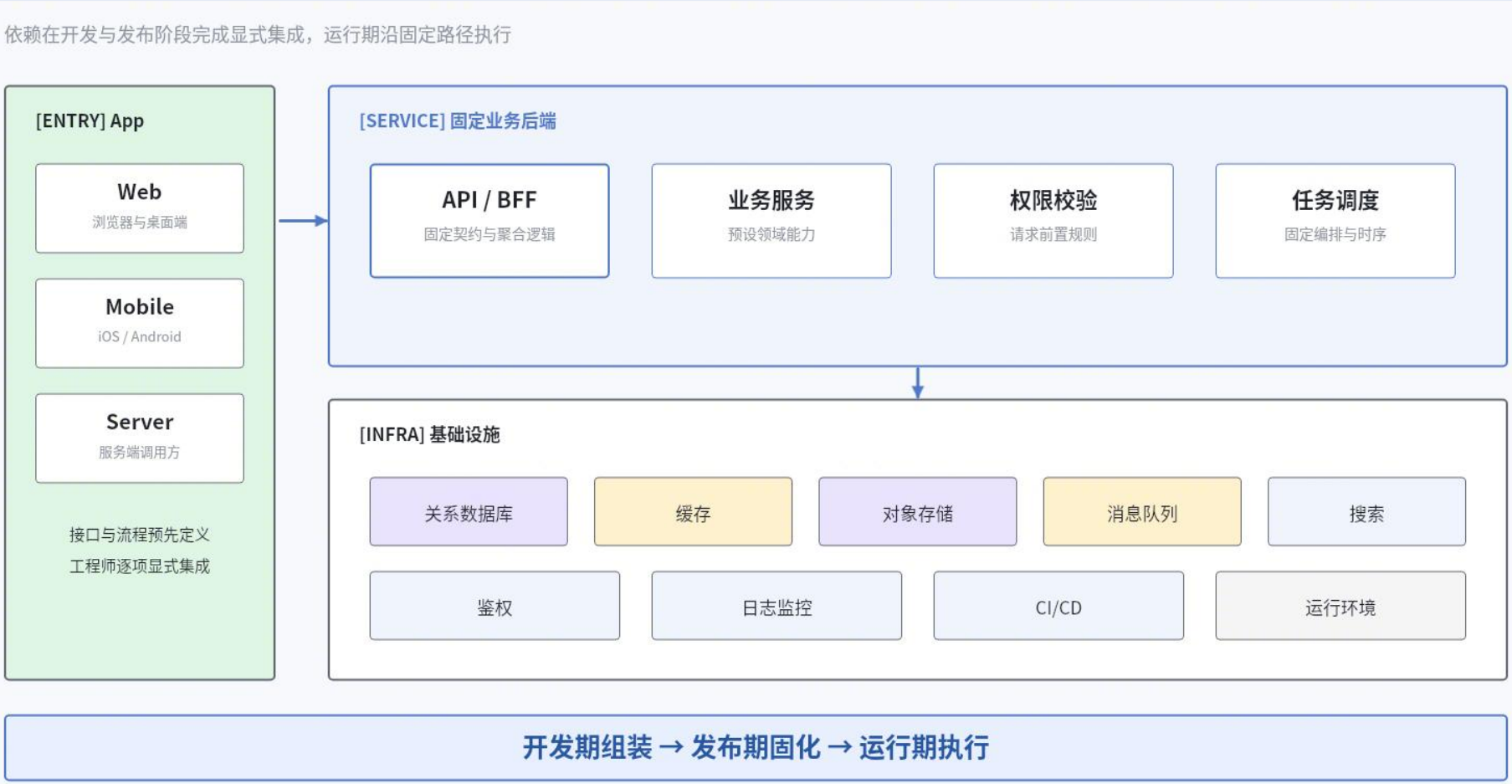
BaaS 在 AI Native 架构中的应用



02

从 App 到 Agent：后端基础设施发生了什么变化

App 依赖的后端基础设施



App 的后端依赖在开发与发布阶段被预先编排

Agent 依赖的后端基础设施



Agent 的后端依赖在运行阶段被动态发现、组合与调用

传统 BaaS 已经解决什么，生产级 Agent 又增加了哪些问题

从“确定性后端能力”走向“概率性自主执行系统”

传统 BaaS：把后端能力变成可靠服务

确定性服务面

1. **身份、租户与权限**
   认证、RBAC、密钥、配额、审计边界
2. **数据、文件与状态存储**
   数据库、对象存储、缓存、备份与一致性
3. **事件、任务与外部集成**
   消息、队列、Webhook、定时任务、第三方 API
4. **运行、弹性与高可用**
   部署、扩缩容、容灾、限流、SLA 与成本治理
5. **安全、合规与可观测**
   日志、指标、追踪、告警、审计与数据隔离

核心承诺：给定输入与配置，系统行为可预测、可复现

范式跃迁

API 调用 → 目标委托

系统开始自己决定

做什么 · 何时做 · 调什么工具

BaaS 仍是地基
但“成功”不再只看 API

生产级 Agent：把“自主行为”纳入工程控制

概率性执行面

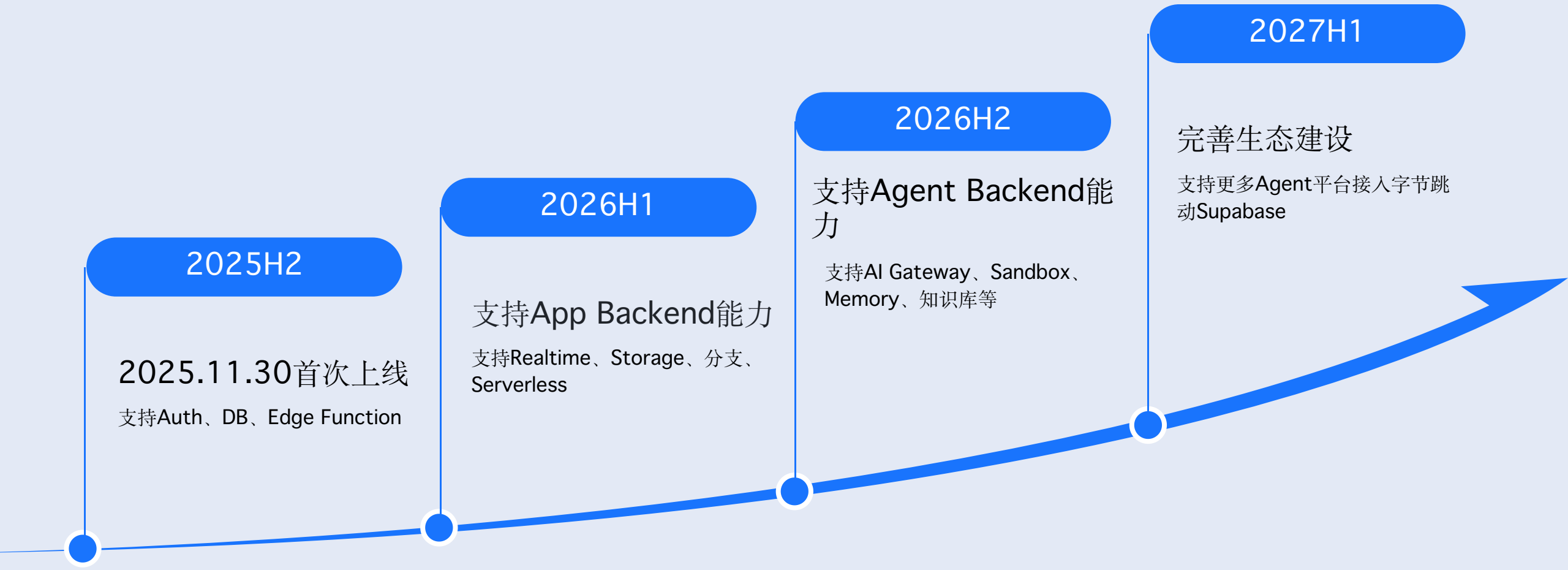
Agent Runtime
感知 → 规划 → 执行
观察 → 反思 → 继续

- **① 非确定性**
  评测、回归、置信边界
  持续评测与回归
- **② 状态与记忆**
  上下文、长期记忆、隔离
  可恢复、可删除、可解释
- **③ 工具与权限**
  最小授权、沙箱、审批
  防注入、越权与误操作
- **④ 长任务可靠性**
  断点续跑、幂等、补偿
  超时、重试、任务取消
- **⑤ 版本与漂移**
  模型与工具持续漂移
  版本锁定、灰度与回滚
- **⑥ 成本与人机协同**
  Token、时延、预算与降级
  接管、申诉、责任归属

核心承诺升级：不仅服务可用，还要让一次“目标执行”可控、可证、可追责

生产级 Agent 对 BaaS 服务增加了 LLM 、Sandbox、记忆库、知识库、向量存储等诉求

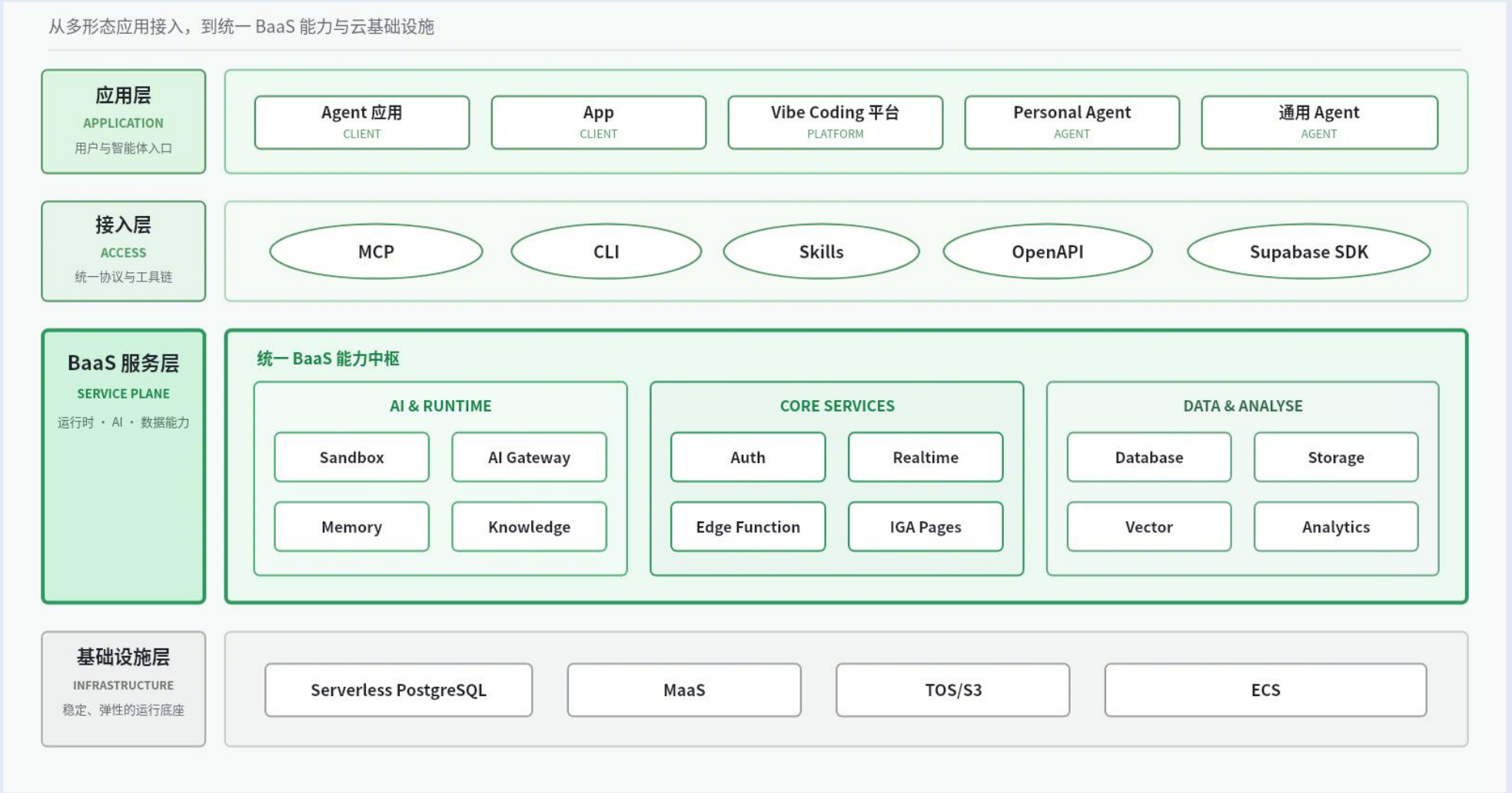
字节跳动 Supabase 从 App Backend 向 Agent-Native 能力演进路径



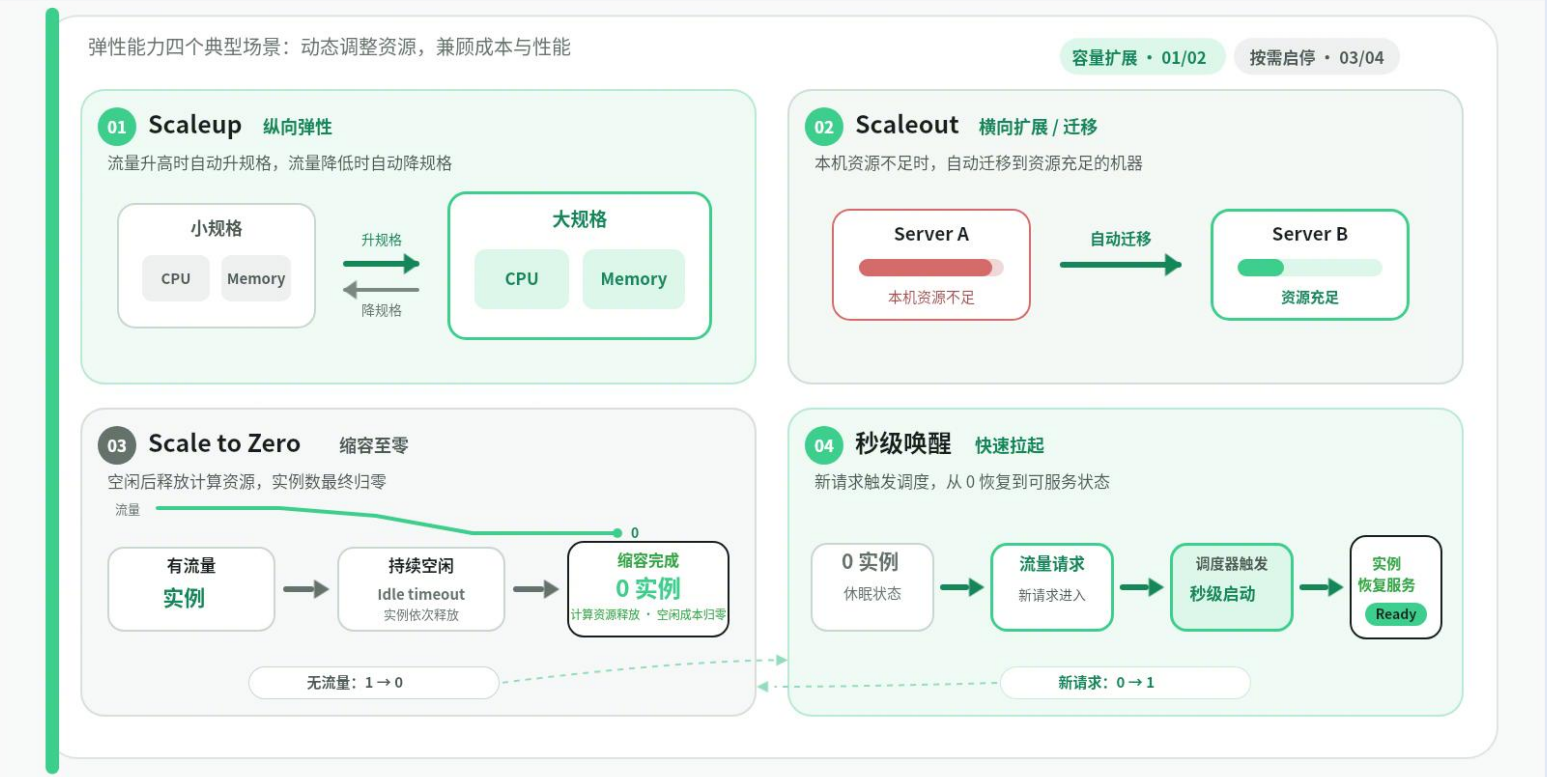
03

字节跳动 Agent-Native BaaS 技术方案

Agent-Native BaaS 整体架构：Agentic Supabase



Serverless： Supabase + Database 全场景实现Serverless



业务痛点：

1. 请求高度离散，低峰空耗与突发扩容同时存在；自建开源版本通常只解决“能部署”，未解决资源利用率问题

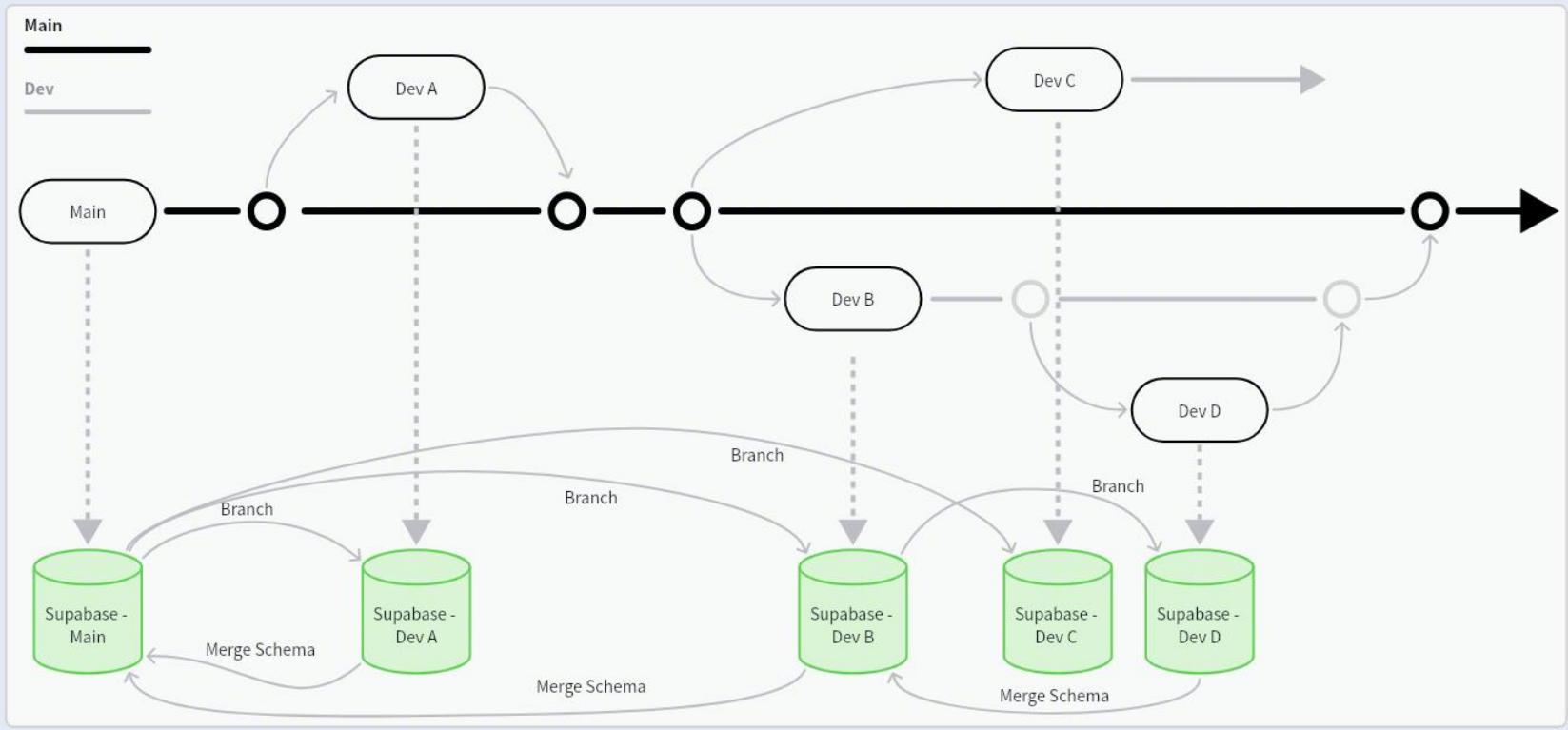
解决方案：

1. Supabase：Auth、Storage、Edge Function等各组件均实现Serverless
2. Database：通过自研存算分离架构，实现DB的Serverless

技术挑战：

1. 实例休眠后，业务首次访问需要先唤醒，唤醒时间会影响业务响应时间
2. 如何实现大规模实例的Serverless弹性与资源调度

Data as Git：像管理代码分支一样管理后端资源



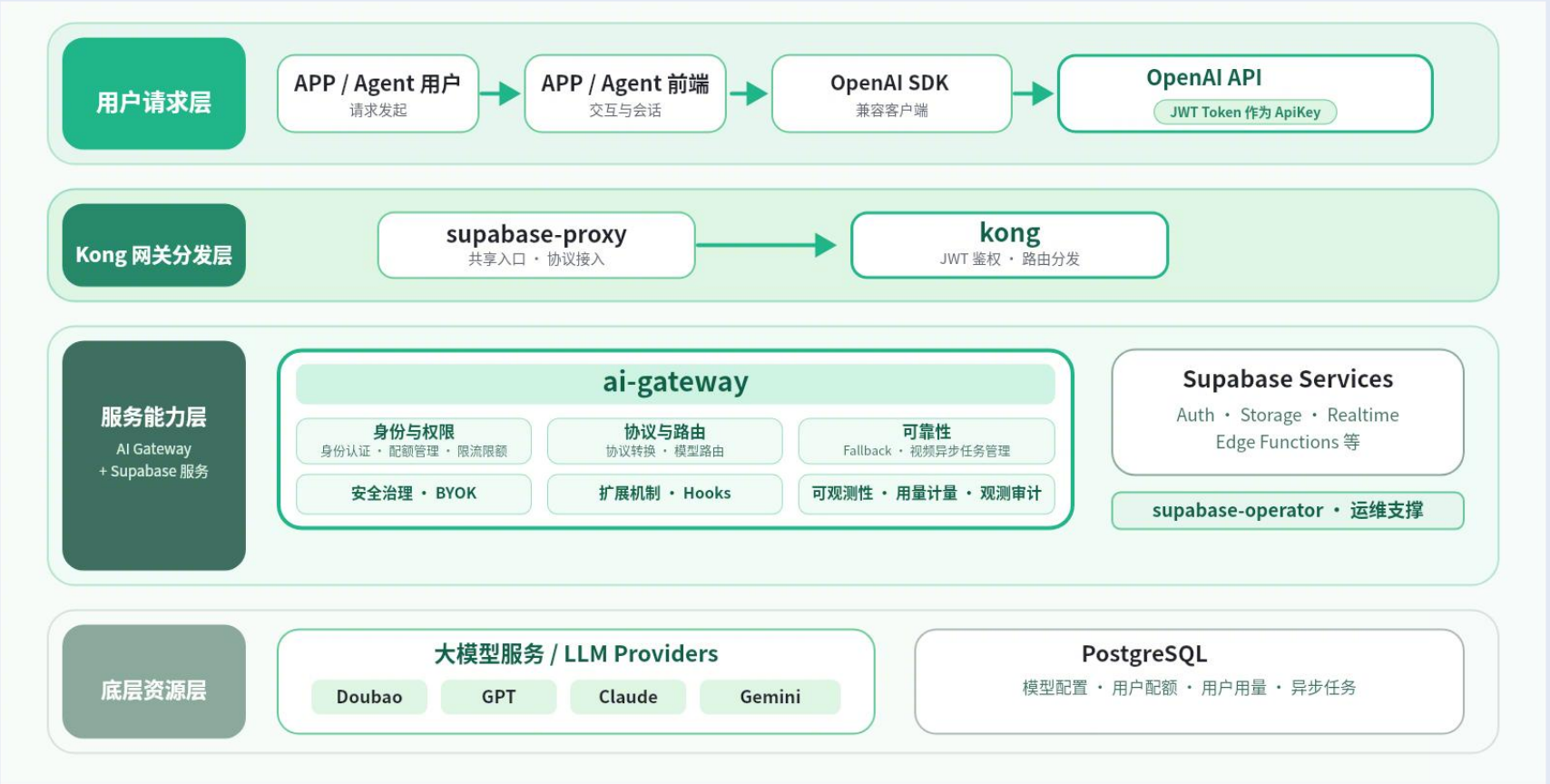
应用场景：

1. 当生产分支上线后，需要对产品进行持续迭代时，可以将线上分支拉出用来开发

1. 当AI生成的最新代码不如历史版本时，可以使用Restore秒级的将Supabase回滚到对应历史时间点，快速的来比对AI生成不同版本代码效果

1. 当数据被误删或想查看历史DB数据时，设置Timetravel时间，可以直接使用SQL查看历史数据

AI Gateway：统一模型入口，智能路由底层资源



业务痛点：

1. 购买、配置模型步骤繁琐，API Key容易泄漏
2. 多模型 API 接口不统一，切换成本高
3. 模型故障后无法降级
4. 缺少用户级Token配额、限流能力

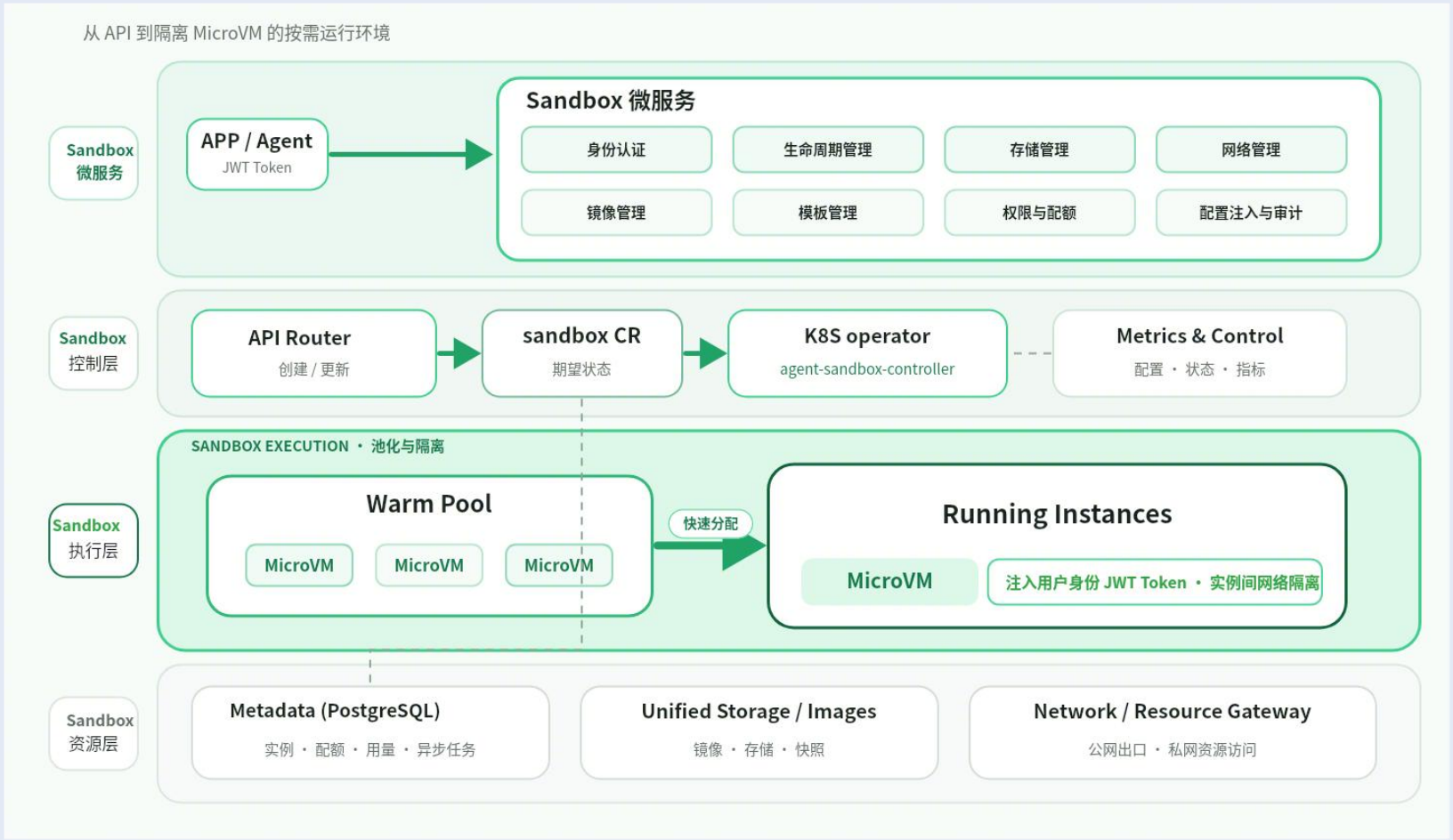
解决方案：

1. 通过与Supabase Auth、DB、Storage协同，完成模型入口统一

技术挑战：

1. 协议转换与Fallback可能掩盖模型差异

Sandbox：Agent的隔离执行环境与Agent Runtime



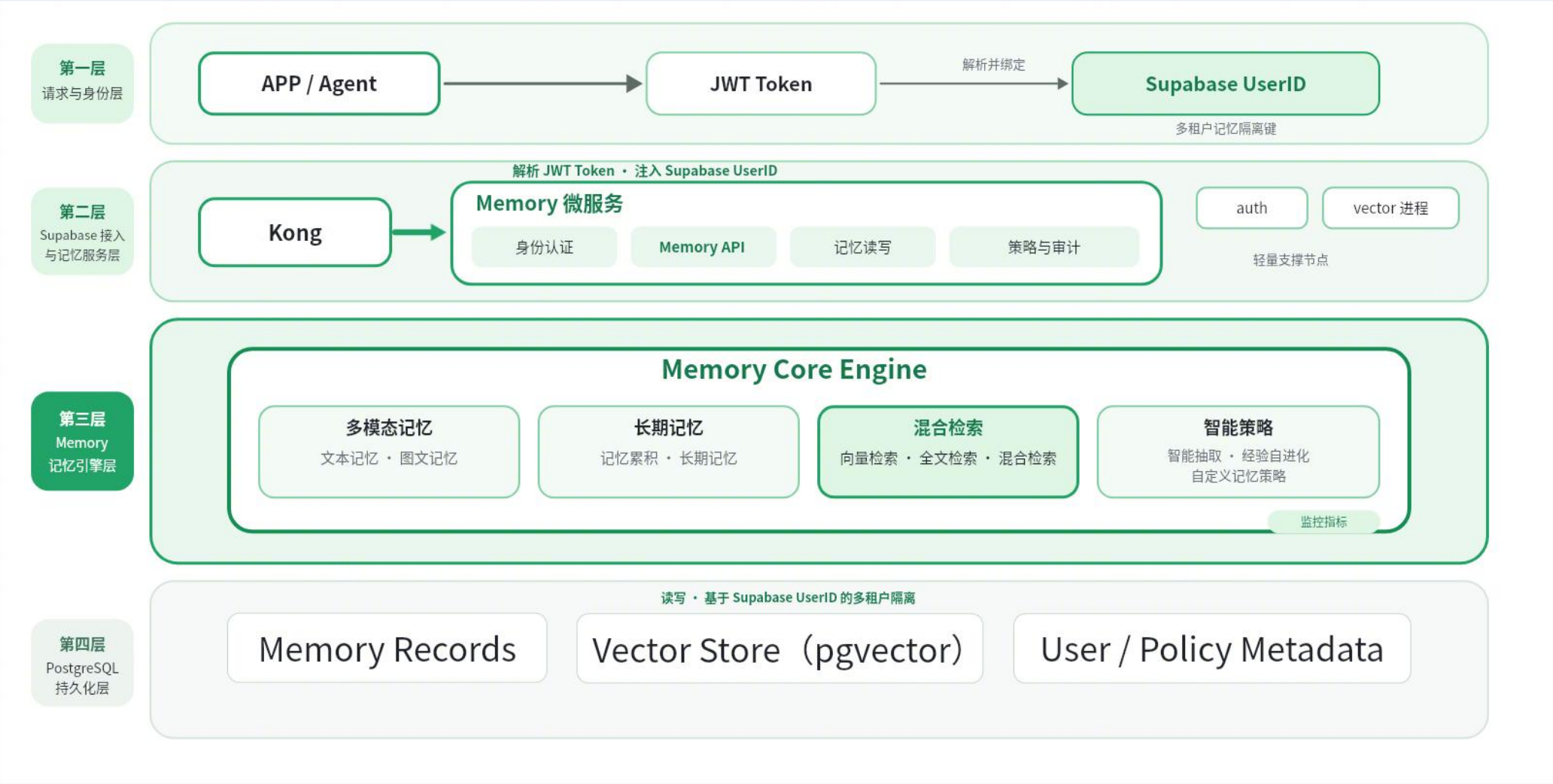
Agent in Sandbox：

Agent 运行在隔离沙箱中，在受控的计算、网络、文件和工具权限范围内自主完成代码执行、文件处理、数据分析等任务，避免对主系统和其他任务造成影响

Agent call Sandbox：

Agent 根据任务需要动态调用沙箱执行特定操作，例如运行代码、处理文件或访问已授权资源；由平台负责调度、权限校验、资源限制和执行结果回传

记忆库：为 Agent 提供身份隔离、长期记忆与混合检索



04

字节跳动 Supabase 的规模化实践

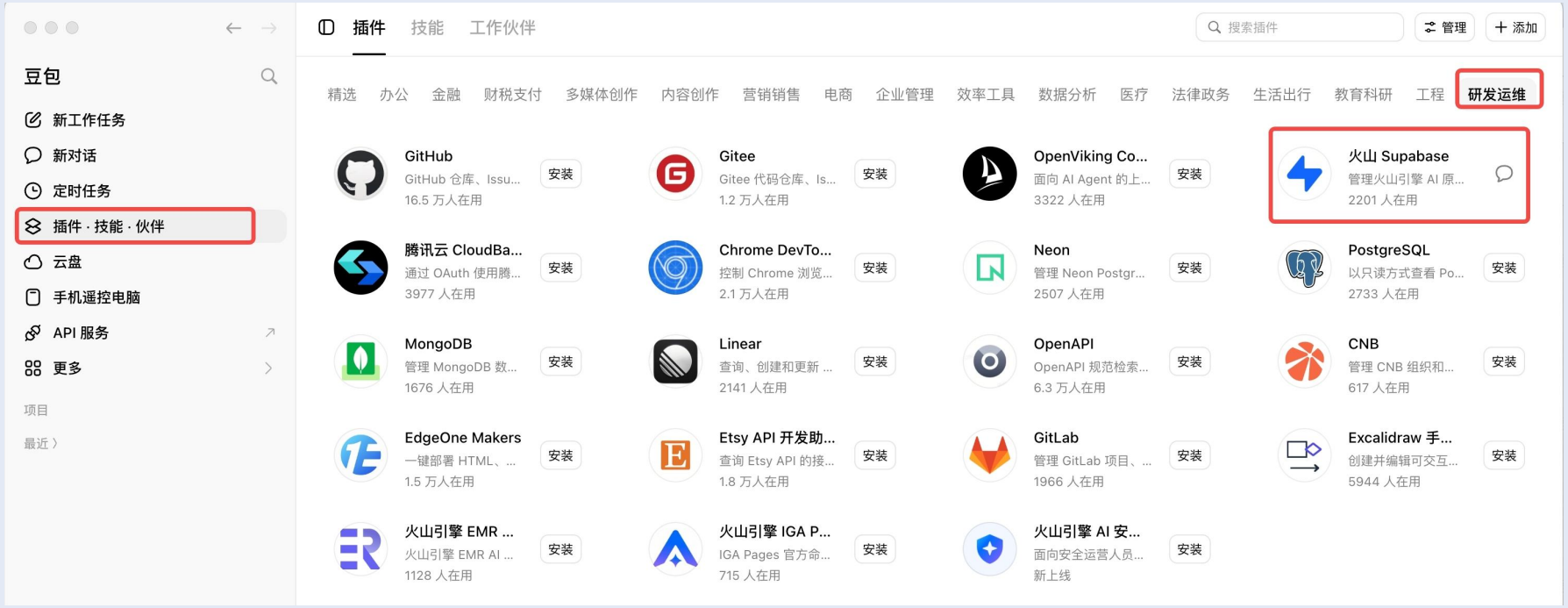
豆包应用生成实践：让非专业开发者获得完整后端并实现生产级应用

场景挑战

非专业开发者用户可以快速使用AI生成前端、后端，但上线部署还需要申请各类云资源与手工配置，这类操作阻塞了应用的快速上线与运行。

技术方案

把生产级默认能力前置到平台，通过CLI/Skills接口暴漏给Coding Agent，使得AI 能从代码生成到部署上线全自动化



豆包应用生成实践：让非专业开发者获得完整后端并实现生产级应用



豆包应用生成实践：让非专业开发者获得完整后端并实现生产级应用





幼升小数学培训班报名系统已完成并部署上线。

公网链接：https://math-registratio-0evfdjv2mf-1nr4jiiyyo.preview.iga-pages.com?iga_token=e9044022a20fb0423635ef5b1f868d11&iga_time=1791356796

系统组成

- 前端报名页：学生姓名、年龄（3-12 校验）、性别、家长电话（11 位手机号校验），提交成功有明确提示
- 数据存储：写入 `supabase-test1` 实例的 `registrations` 表（含 RLS 安全策略 —— 匿名可提交、不可读取他人信息，保护报名隐私）
- 部署：火山 IGA Pages，已绑定 Supabase 分支

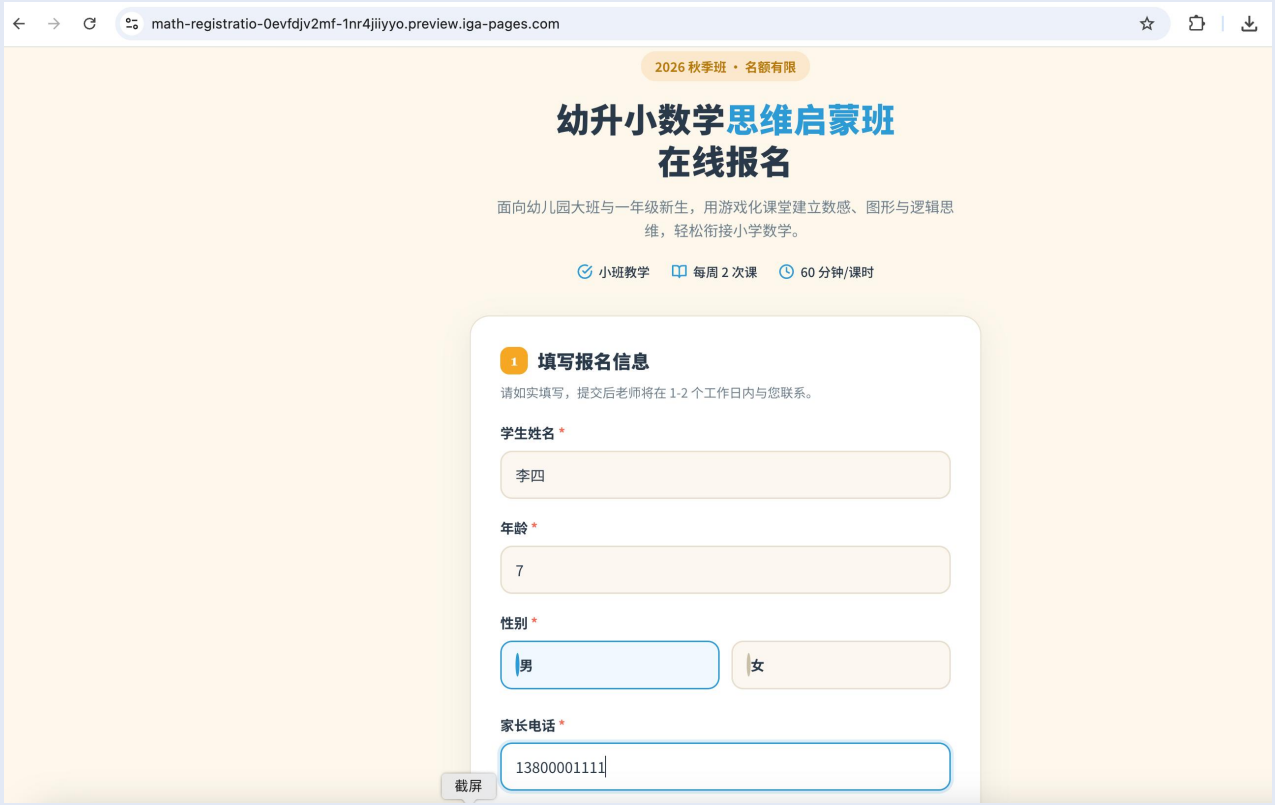
已验证：模拟提交返回成功（HTTP 201），数据正常落库，测试数据已清理。

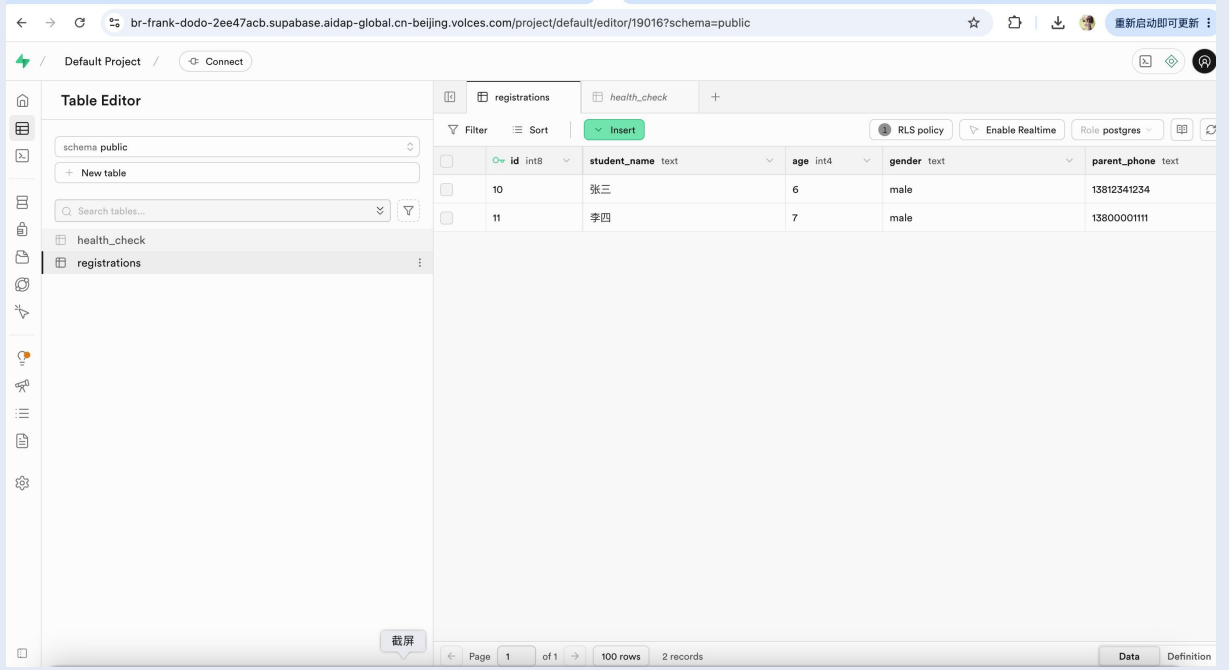
两点提醒：

- 该预览链接带一次性访问令牌、会话约 3 小时有效；失效或换设备打不开时告诉我，我随时重新生成新链接。如需长期稳定域名，可再配置正式域名。
- 报名数据目前只能通过数据库侧查看（控制台或查询工具），如需在页面上加 "报名管理后台"（查看 / 导出名单），也可以继续帮你做。

源码保存在 `math-registration-site/index.html`，改完文案或字段后告诉我即可重新部署。

豆包应用生成实践：让非专业开发者获得完整后端并实现生产级应用

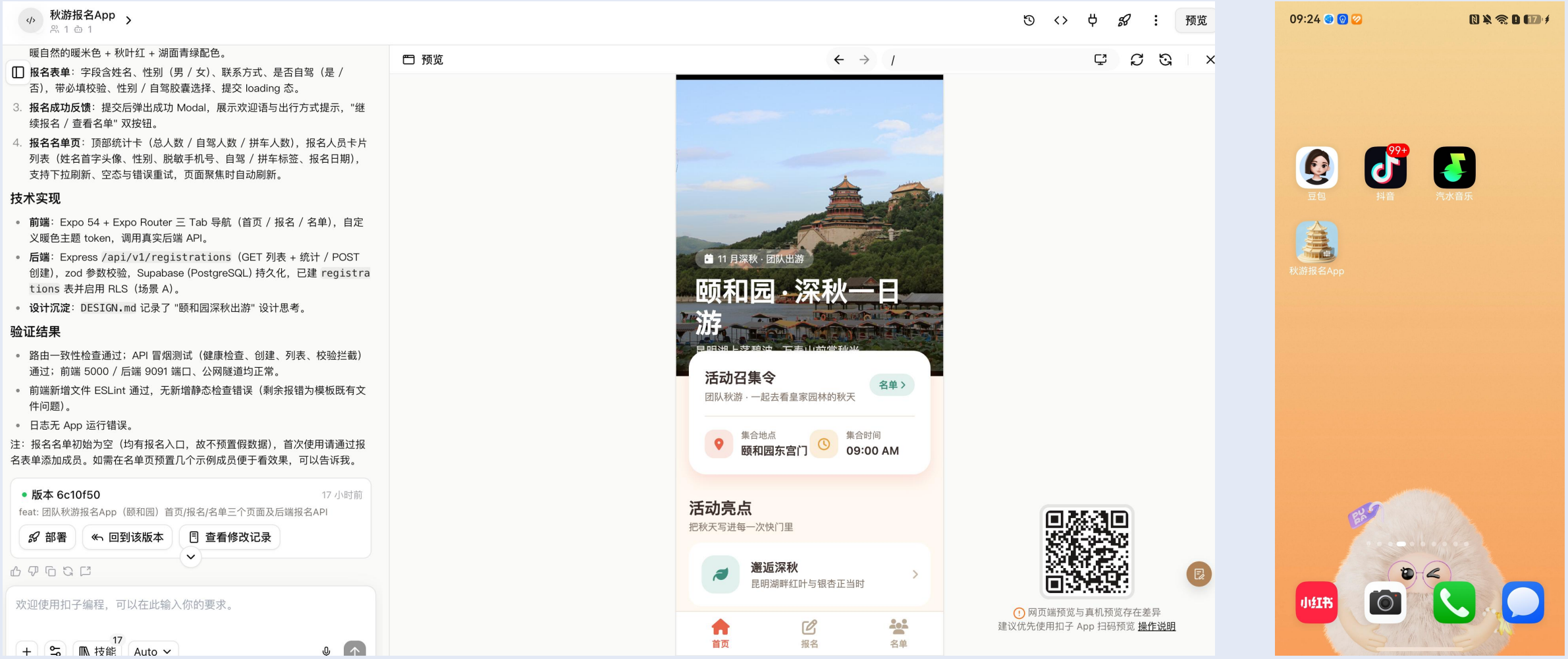




Coze 编程实践：使用分支支撑前后端并行试错，实现开发与生产之间的隔离



Coze 编程实践：使用分支支撑前后端并行试错，实现开发与生产之间的隔离



Coze 编程实践：使用分支支撑前后端并行试错，实现开发与生产之间的隔离

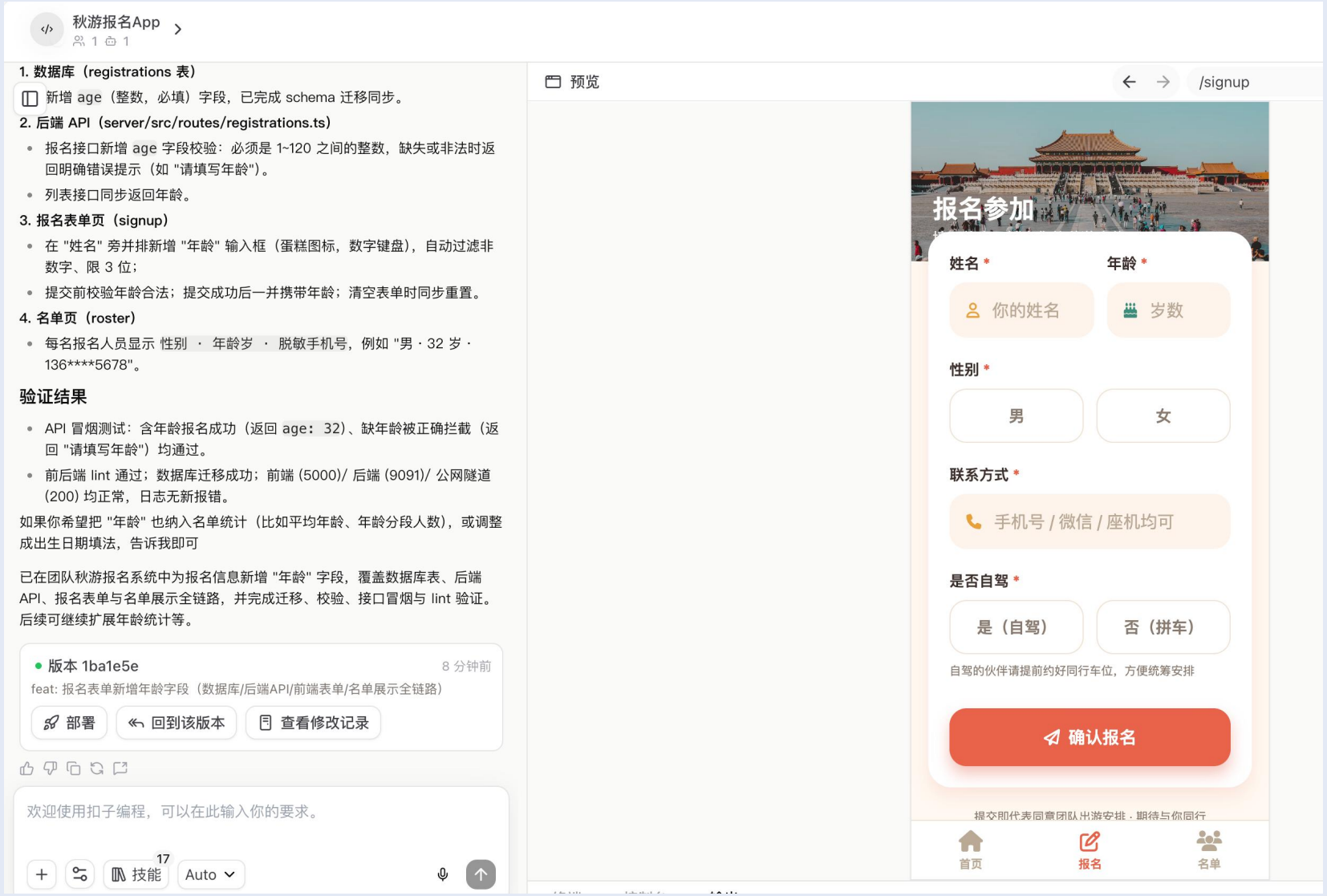




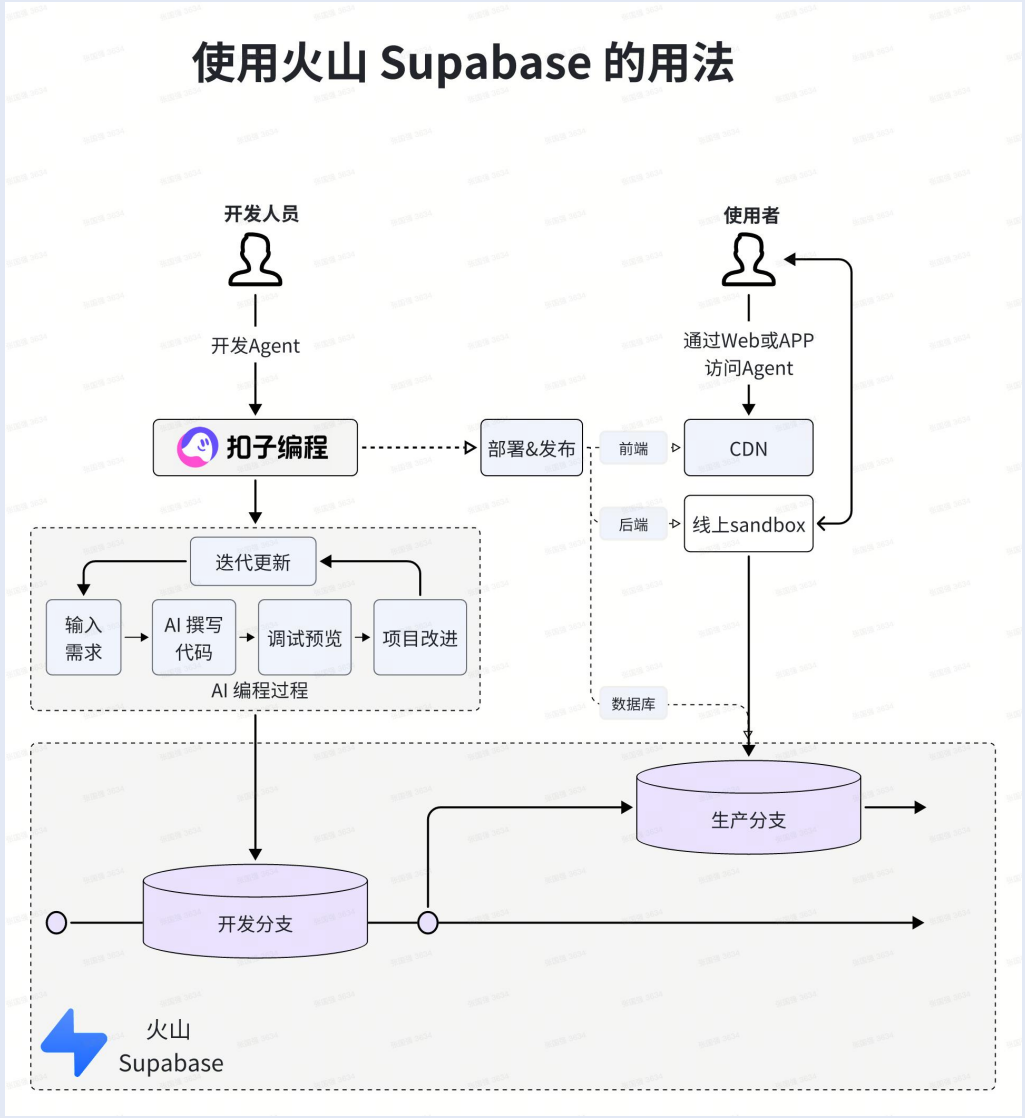




Coze 编程实践：使用分支支撑前后端并行试错，实现开发与生产之间的隔离

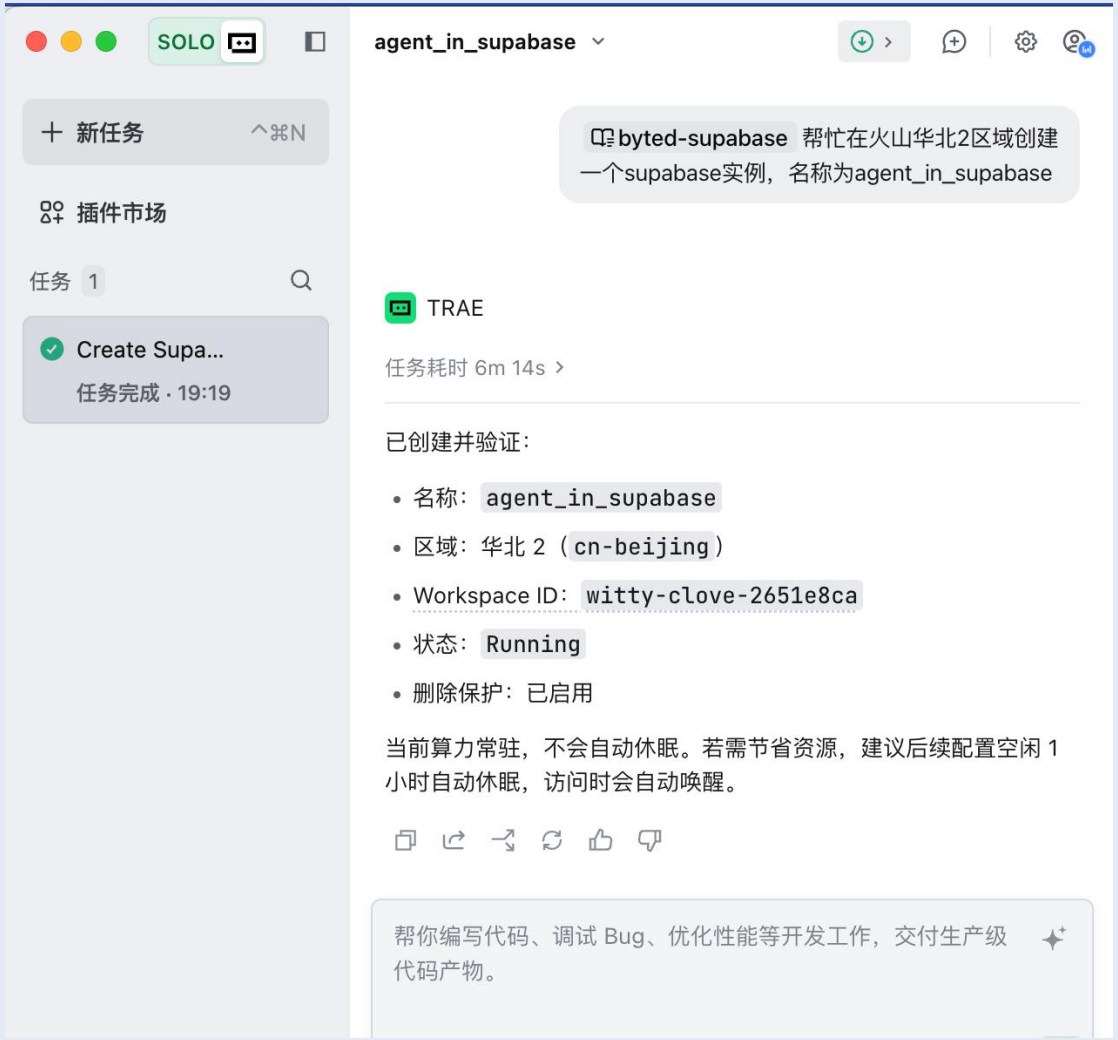


Coze 编程实践：使用分支支撑前后端并行试错，实现开发与生产之间的隔离





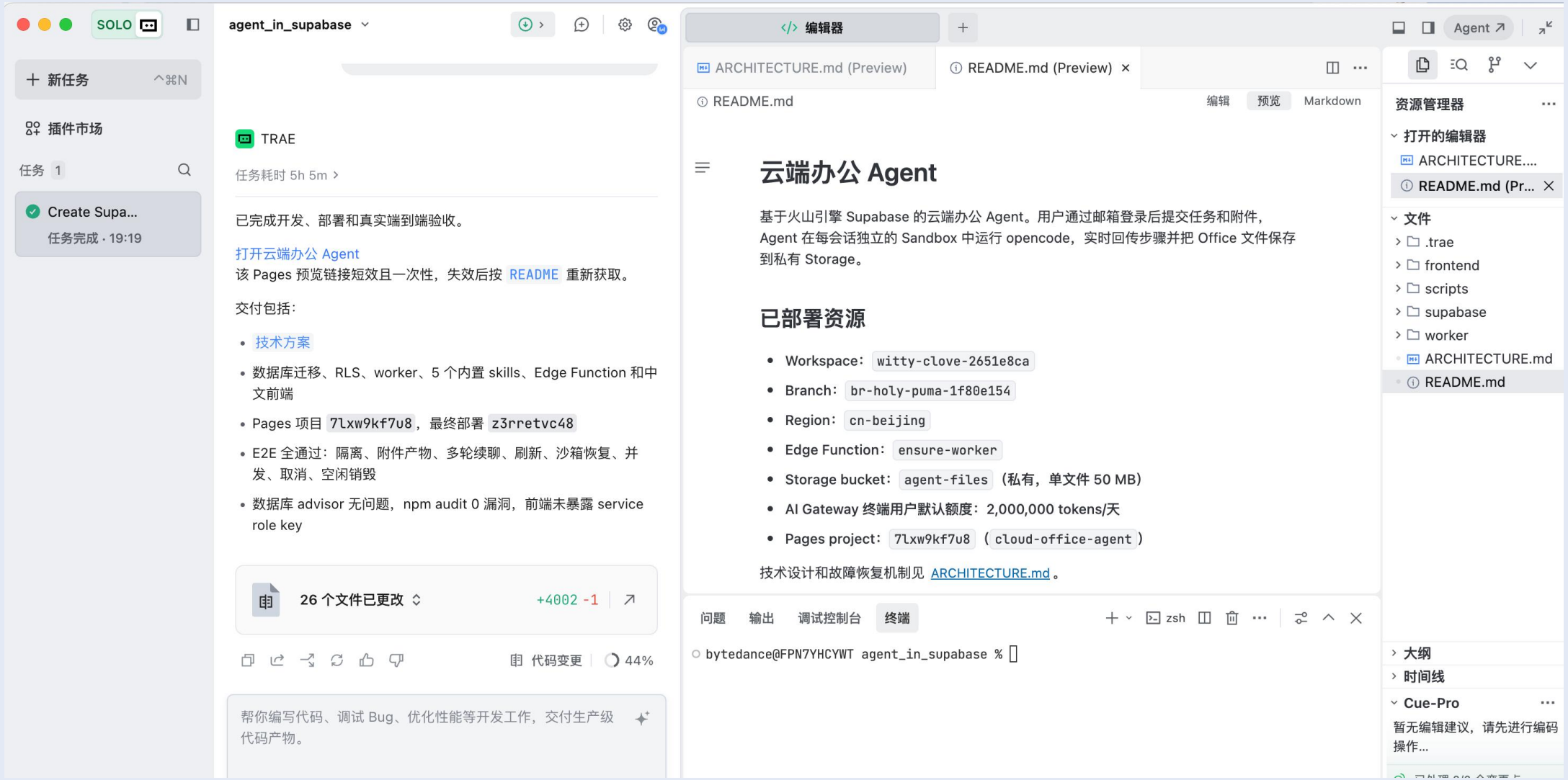
Trae 编程实践：通过 skills 实现 Agent 的一键部署与上线



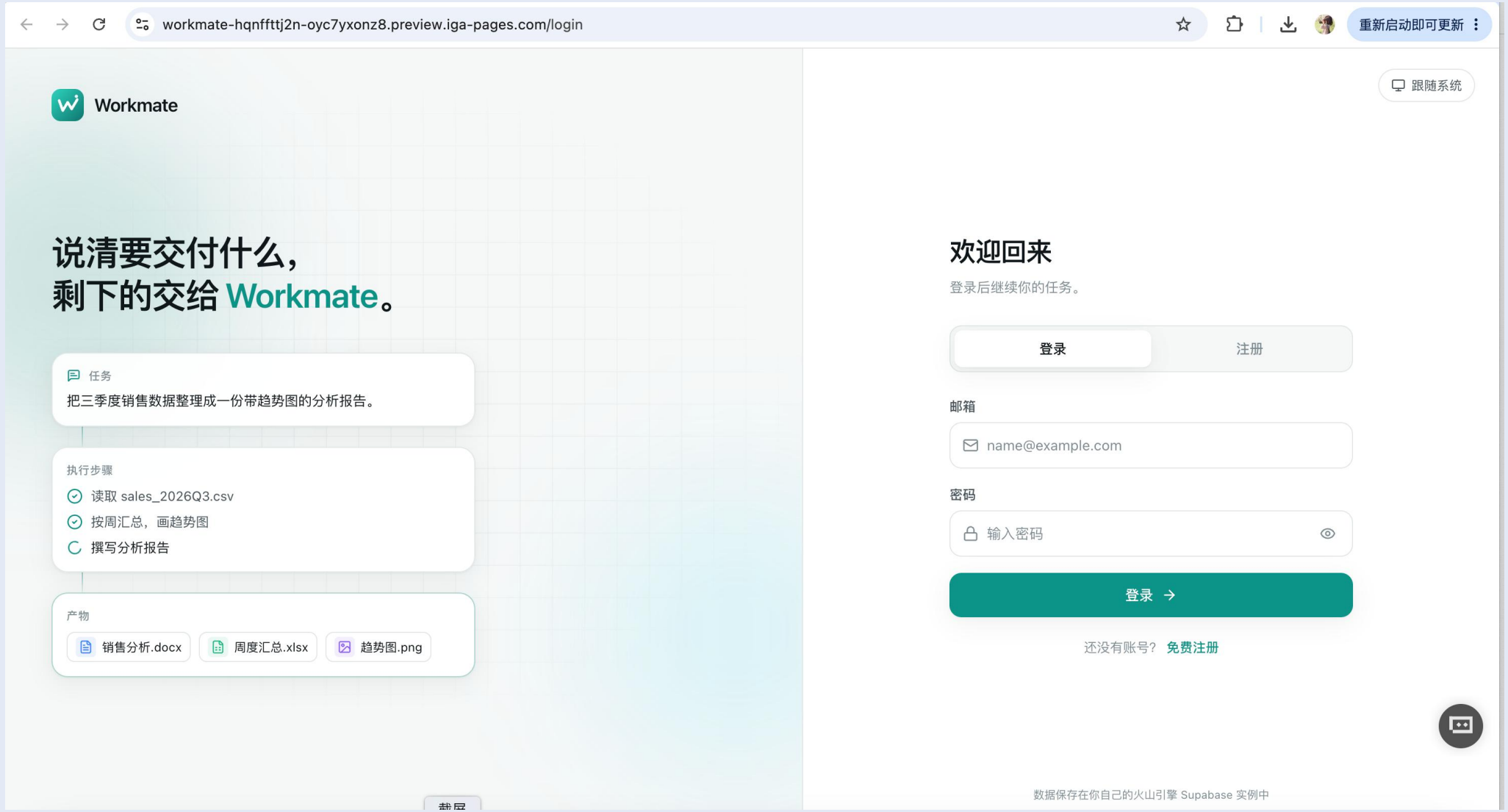
prompt:

参考 /byted-supabase skills，基于上面创建的 supabase workspace，开发一个云端办公 Agent App：用户登录后提交办公任务和附件，每个会话启动一个 Sandbox worker，调用 opencode 自动写脚本、跑代码、生成文件；前端通过 Realtime 实时展示步骤、工具调用和产物，支持文件预览下载、多轮续聊、任务取消、会话管理、用户隔离和断线恢复。

Trae 编程实践：通过 skills 实现 Agent 的一键部署与上线



Trae 编程实践：通过 skills 实现 Agent 的一键部署与上线



Trae 编程实践：通过 skills 实现 Agent 的一键部署与上线



Trae 编程实践：通过 Supabase skills 实现 Agent 的一键部署与上线



某房地产行业用户：从「能生成」到「能长期用」

客户案例　行业：居住服务 | 服务对象：一线经纪人与 职能同学

瓶颈转移到底座

- 生成页面后，应用仍需可靠数据读写、组织与门店维度权限、文件存储与实时协作
- 应用规模从几十个增长到数万级，必须统一治理实例、权限边界与版本
- 底座撑不住，员工做的工具就停在 Demo，进不了日常工作流

Supabase 承接的部分

- 数据库表自动转标准接口，减少接口开发与联调
- Auth + RLS 把组织、城市、门店、角色的数据边界写进规则
- Realtime 让成交、客户记录、履约状态与看板数据秒级同步

业务结果

- Edge Function 连接内部 AI 大模型、企业微信与业务系统
- 门店经营平台 7 天做出 MVP
- 某省会平台成功复制到 13 座城市

约 3.4 倍
新集群承载应用量对比老集群（6.432 万 对比 1.867 万）

79%
新集群应用活跃率，老集群 56%（高 23 个百分点）

7 天
门店经营平台从想法到 MVP

承载量与活跃率同时提升，才说明底座真的撑住了

应用数量涨到 3.4 倍的同时活跃率还从 56% 提升到 79%——这说明不是「堆了更多 Demo」，而是员工做的工具真的进入了日常工作流。

某教育行业用户：从内部建站，到 Agent 托管平台

客户案例　行业：教育科技 | 场景：产品研发、内容生产、营销、教学教研与职能流程

业务痛点

- 活动页、问卷、招生工具时效性强，传统研发排期以周计，错过窗口即失去价值
- 对话式生成只出前端页面，后端逻辑、部署、鉴权仍要研发补齐，提效只完成一半
- 课程、班级、校区、角色的读写权限不能靠临时代码兜底

选择 Supabase 的理由

- 大模型对 Supabase 理解充分，生成代码可直接跑通
- 常见后端需求覆盖完整，DB、Auth、Storage 一次到位
- 具备按需计费与空闲停止计费能力，支撑大量低频应用

交付链路

- 对话提需求 → Agent 生成代码
- Supabase 承接后端能力
- 预览部署，1 Session = 1 Sandbox 保证会话隔离与长期存活

数周 → 几分钟

教学工具创意的落地周期（原为数周至数月）

降低约 1/3

HR 面试预约与反馈系统带来的沟通成本降幅

1 : 1

1 Session = 1 Sandbox，会话隔离与长期存活

大模型「懂」你的后端，生成代码才能直接跑通

用户选择我们的第一个理由不是功能多，而是大模型对 Supabase 的理解足够充分——这是 100% 兼容开源用法带来的直接收益。

05

未来展望

技术架构总结

01 使用方

应用数量多、单个访问量低，但每个都需要一套完整后端

- AI Agent 应用 — 记忆 / 任务 / RAG 检索
- AI Coding 平台 — 生成物即得可用后端
- 内部工具 / 后台 — 表单 / 看板 / 运营系统
- MVP / 原型验证 — 先跑通再决定投入

02 接入层

- 火山控制台 — 可视化建库建表
- CLI + 本地 MCP — 脚本化 / Agent 调用
- Skills — byted-supabase
- OpenAPI — 平台方批量托管
- Supabase SDK — 前端直连，开源同款

03 开箱即用的 BaaS 服务层

九项能力装好即用，不用自己搭

- Database + 自动 API — PostgreSQL / PostgREST / GraphQL
- Auth + RLS — SSO / JWT / 行级权限
- Storage + Vector — 对象存储 / 上传下载 / 权限同源
- Realtime — 基于 WAL 的变更订阅推送
- Edge Function — TS / Python，仅可访问公网
- Agent Sandbox — AIO 沙箱 / Codex 长短周期任务
- 前端部署 — SSR 渲染 / 流水线 / 自定义域名
- AI Gateway — 免 key 接模型 / 鉴权 / 流控 / 审计
- 知识 + 记忆 — Context 组件 / 上下文工程

04 Serverless PostgreSQL 内核底座

项目 → Workspace → Branch → 算力 CU

项目	Workspace	Branch	算力 CU
一个业务一个项目	访问地址与算力设置	自带 main，可拉分支	伸缩区间按需设定

按需伸缩 · 无访问自动休眠

算力随负载伸缩，没有访问时自动休眠，低峰不空耗

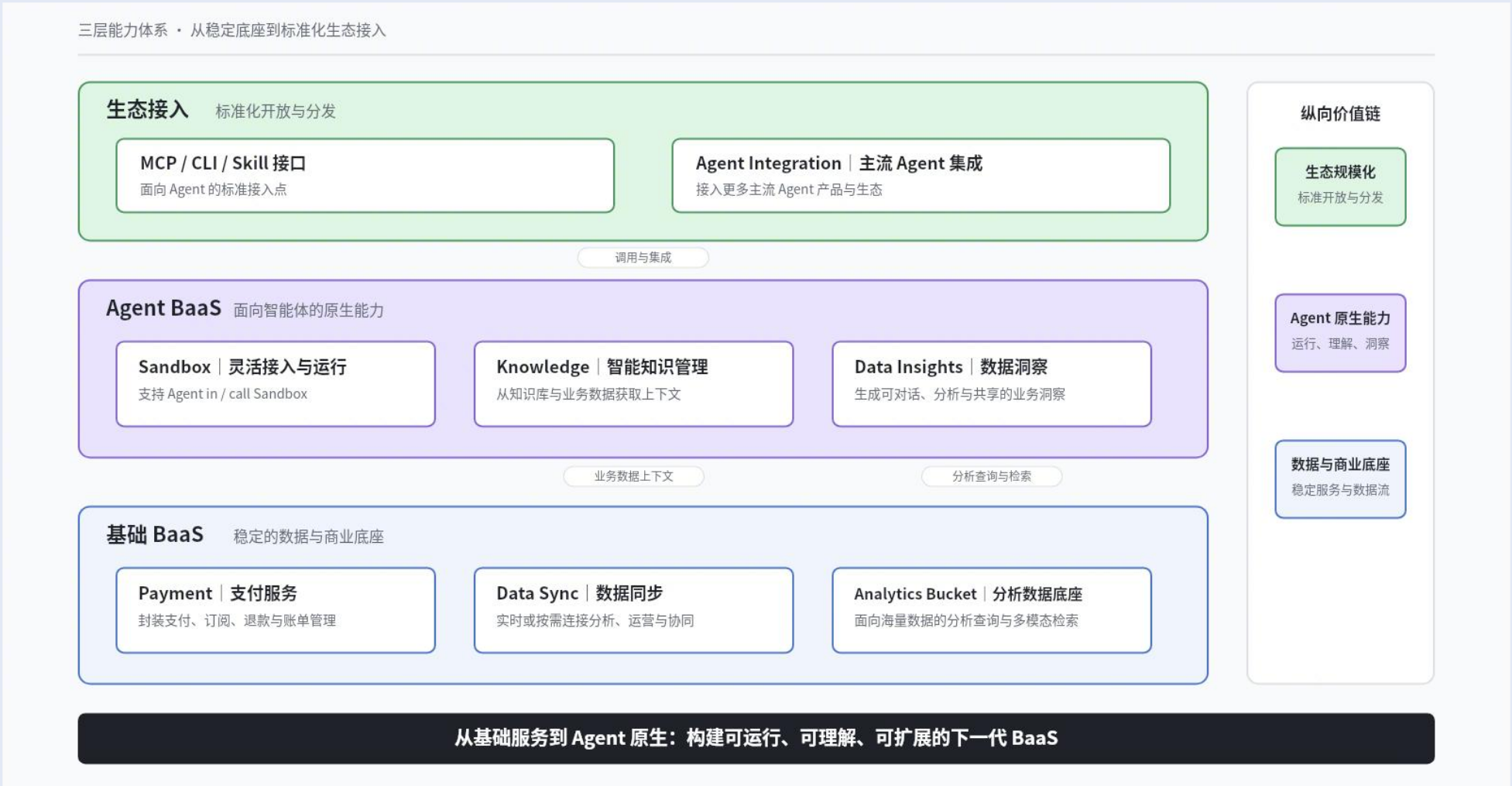
长尾应用持有成本贴近零

海量低频应用的持有成本趋近于零，长尾场景才能规模化

Data as Git

Branch 隔离环境，Timetravel / PITR 回滚误操作

未来演进方向：面向 Agent 的下一代 BaaS 架构



AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

